

Kapitel 3: Grundlagen der Programmierung

3.4. Übungsaufgaben

Aufgabe 1

Nachfolgend wird an mehreren Beispielen die Deklaration, Initialisierung sowie Neu- und Wiederbelegung einer Variablen gezeigt.

Eure Aufgabe ist es, den Wert und den Datentyp der benannten Variablen am Ende des kleinen Programms zu bestimmen.

Versucht zunächst, dies **ohne** Ausführung des Codes zu lösen, also nur durch zeilenweises Nachvollziehen des Programms.

Notiert eure Lösungen jeweils dort, wo steht: ... eure Antwort hier Ändern könnt ihr die Texte durch einen Doppelklick auf die entsprechende Stelle.

```
number = 1
number = number + 5
number = number * 2
number = number % 12
print(number, type(number))
```

0 <class 'int'>

Welchen Wert und Datentyp hat die Variable `number` ?

- Wert: 0
- Datentyp: int

```
number = '1.23'
number = float(number)
text = int(number)
neuer_text = text * 5
neuer_text = str(neuer_text)

print(neuer_text, type(neuer_text))
```

5 <class 'str'>

Welchen Wert und Datentyp hat die Variable `neuer_text` ?

- Wert: 5
- Datentyp: str

```
irgendwas = 'Hallo Schüler(in)'
wie_gehts = ' - wie geht es dir heute?'
text = wie_gehts + irgendwas
text = text * 2
text = 2
text = irgendwas * text

print(text, type(text))
```

Hallo Schüler(in)Hallo Schüler(in) <class 'str'>

Welchen Wert und Datentyp hat die Variable `text` ?

- Wert: `Hallo Schüler(in)`
- Datentyp: `str`

```
nutzereingabe = 2
nutzereingabe = nutzereingabe**2
nutzereingabe = nutzereingabe // 3
nutzereingabe = nutzereingabe * 10
ausgabe = 'Das Ergebnis ist: ' + str(nutzereingabe)

print(ausgabe, type(ausgabe))
```

Das Ergebnis ist: 10 <class 'str'>

Welchen Wert und Datentyp hat die Variable `ausgabe` ?

- Wert: `Das Ergebnis ist: 10`
- Datentyp: `str`

Aufgabe 2

In der folgenden Tabelle sind verschiedene Werte vorgegeben.

Anhand dieser Werte sollt ihr ausprobieren, wie sich verschiedene Typumwandlungen auswirken. Dazu könnt ihr die folgende Programmstruktur als Hilfsmittel nutzen.

```
wert_vorher = 1.9 # <-- Spalte 1 der Tabelle
print(wert_vorher, type(wert_vorher))

print('---')

wert_nachher = int(wert_vorher) # <-- an der Stelle muss immer die Typisierungsfunktion geändert werden
print(wert_nachher, type(wert_nachher))
```

Vorgehensweise:

- Ihr beginnt in der ersten Zeile der Tabelle. Dort steht in der Spalte **Wert (vorher)** der Wert **1.9**. Im oberen Programm weist ihr der Variable **wert_vorher** genau diesen Wert zu.
- Außerdem enthält die Tabelle die gewünschte Typisierungsfunktion. Diese wird in der vierten Codezeile im obigen Codeblock verwendet (der Kommentar gibt einen Hinweis).
- Führt ihr das Programm aus, so erhaltet ihr zunächst den Wert und Datentyp **vor der Typisierung** und anschließend den Wert und Datentyp **nach der Typisierung**.
- Tragt eure Ergebnisse in die Tabelle ein. Sollte es zu Besonderheiten kommen, wie direkt in der ersten Zeile der Tabelle, tragt diese in der letzten Spalte ein.
- Änderungen in der Tabelle könnt ihr vornehmen, indem ihr doppelt auf die Tabelle klickt.

Wert (vorher)	Datentyp (vorher)	Typisierungsfunktion	Wert (danach)	Datentyp (danach)	Besonderheit(en)
1.9	float	int()	1	int	Nachkommastellen werden abgeschnitten. Es wird nicht gerundet.
42	int	int()	42	int	Wert und Datentyp bleiben unverändert.
42	int	float()	42.0	float	Die Ganzzahl wird zur Gleitkommazahl.
1.2	float	float()	1.2	float	Wert und Datentyp bleiben unverändert.
42	int	str()	'42'	str	Die Zahl wird als Zeichenkette dargestellt.
1.2	float	str()	'1.2'	str	Die Zahl wird als Zeichenkette dargestellt.
'12.34'	str	int()	Fehler: ValueError	—	Die Zeichenkette stellt keine Ganzzahl dar.
'12.34'	str	float()	12.34	float	Die Zeichenkette wird als Gleitkommazahl interpretiert.
'112'	str	int()	112	int	Die Zeichenkette wird als Ganzzahl interpretiert.
'112'	str	float()	112.0	float	Auch ohne Dezimalpunkt ist die Umwandlung möglich.
'1'	str	bool()	True	bool	Nicht leere Zeichenketten ergeben <code>True</code> .
'0'	str	bool()	True	bool	Auch <code>'0'</code> ist eine nicht leere Zeichenkette und ergibt <code>True</code> .
True	bool	int()	1	int	<code>True</code> wird zu <code>1</code> .
False	bool	int()	0	int	<code>False</code> wird zu <code>0</code> .
True	bool	float()	1.0	float	<code>True</code> wird zu <code>1.0</code> .
False	bool	float()	0.0	float	<code>False</code> wird zu <code>0.0</code> .
None	NoneType	str()	'None'	str	Es entsteht die Zeichenkette <code>'None'</code> .
None	NoneType	bool()	False	bool	<code>None</code> gilt als falsch und ergibt <code>False</code> .
None	NoneType	int()	Fehler: TypeError	—	<code>None</code> kann nicht in eine Ganzzahl umgewandelt werden.

Aufgabe 3

Implementiere ein Programm, das die folgenden Funktionalitäten erfüllt:

- Die Nutzerin oder der Nutzer soll aufgefordert werden, eine Zahl einzugeben.
- Diese Eingabe soll zunächst in einer Variablen gespeichert werden.
- Aus der Eingabe soll die Quadratzahl berechnet werden.
- Die Ausgabe soll in etwa so aussehen: 'Die Quadratzahl deiner eingegebenen Zahl **ZAHL** ist **ERGEBNIS** .' Die Platzhalter sollen durch die tatsächlich eingegebenen und berechneten Werte ersetzt werden.

```
# HIER IST PLATZ FÜR EURE LÖSUNG! :)

zahl = input('Gib eine Zahl ein: ')
zahl = float(zahl) # die Aufgabenstellung ist hier unkonkret, int wäre daher auch möglich, aber float ist flexibler
quadrat = zahl ** 2
print(f'Die Quadratzahl deiner eingegebenen Zahl {zahl} ist {quadrat}')
```

Aufgabe 4

Implementiere einen kleinen Taschenrechner für die Grundrechenarten: Dazu sollen die Nutzerin oder der Nutzer zwei Zahlen a und b eingeben. Das Programm gibt dann die Summe $a + b$, die Differenz $a - b$, das Produkt $a \cdot b$ und den Quotienten $a : b$ aus.

```
# HIER IST PLATZ FÜR EURE LÖSUNG! :)

a = input('Gib eine Zahl a ein: ')
b = input('Gib eine Zahl b ein: ')

a = float(a)
b = float(b)

sum = a + b
dif = a - b
prod = a * b
quot = a / b

print(f'{a} + {b} = {sum}')
print(f'{a} - {b} = {dif}')
print(f'{a} * {b} = {prod}')
print(f'{a} / {b} = {quot}')
```

```
5.0 + 3.0 = 8.0
5.0 - 3.0 = 2.0
5.0 * 3.0 = 15.0
5.0 / 3.0 = 1.6666666666666667
```

Aufgabe 5

Implementiere ein Programm, das Eingaben, Typumwandlungen und Ausgaben miteinander verbindet:

- Frage nach dem Namen einer Person.
- Frage nach einer Lieblingszahl und wandle die Eingabe in eine ganze Zahl um.
- Berechne das Doppelte dieser Lieblingszahl.
- Gib einen vollständigen Satz aus, der den Namen, die Lieblingszahl und das Doppelte enthält.

Achte auf passende Variablennamen und darauf, dass Eingaben zunächst als Text gespeichert werden.

```
# HIER IST PLATZ FÜR EURE LÖSUNG! :)
```

```
name = input('Gib deinen Namen ein: ')
```

```
lieblingszahl = input('Gib deine Lieblingszahl ein: ')\nlieblingszahl = float(lieblingszahl)
```

```
doppelt = lieblingszahl * 2
```

```
print(f'Hallo {name}, deine Lieblingszahl ist {lieblingszahl}. Das Doppelte davon ist {doppelt}.')
```

LearningView.org



